DAY - 5 Documentation

🟢 IoT — Friday
Topic: GPS + IMU Firmware Mini Demo
Continue directly from Thursday.
Since there is no physical hardware, the demo will be based on simulation/test data.

Task 1 — GPS
Show the GPS work you started earlier:
• Recorded NMEA data
• Identify GGA / RMC
• Parse:
○ Latitude
○ Longitude
○ Time
○ GPS status/fix
• Print the result through Serial Monitor
Clearly mention:
Recorded/Simulated GPS data — NOT LIVE GPS

Task 2 — IMU
Continue Thursday's simulated IMU work.
Show:
AX
AY
AZ
GX
GY
GZ
and:

Acceleration Magnitude
$\sqrt{(AX^2 + AY^2 + AZ^2)}$
Show at least 3 test conditions:
• Stationary
• Tilt/rotation
• Movement
Again:
SIMULATED / TEST DATA

Task 3 — Explain your firmware flow
You should be able to explain this simple flow:
Input Data
↓
ESP32 Firmware
↓
Parse / Process
↓
GPS + IMU Values
↓

Serial Output

Friday output
- GPS code
- IMU code
- Serial screenshots
- Sample outputs
- Short README
- What worked
- What couldn't be tested because hardware wasn't available

Friday is demo + understanding, not another huge task.

```
#include <Arduino.h>
#include <TinyGPS++.h>
#include <math.h>

TinyGPSPlus gps;

const char *deviceId = "CRASH_ESP32_01";

const char *nmeaGGA =
   "$GPGGA,123519,4807.038,N,01131.000,E,1,08,0.9,545.4,M,46.9,M,,*47\r\n";

const char *nmeaRMC =
   "$GPRMC,123519,A,4807.038,N,01131.000,E,022.4,084.4,230394,003.1,W*6A\r\n";

struct LocationData
{
   String deviceId;
   double latitude;
   double longitude;
   String timestamp;
   bool gpsStatus;
   int fixQuality;
   int satellites;
};

LocationData locationData;

void feedNMEA(const char *sentence)
{
   while (*sentence != '\0')
   {
      gps.encode(*sentence);
      sentence++;
   }
}

String getGPSTimestamp()
{
```

```
  if (gps.date.isValid() && gps.time.isValid())
  {
    char buffer[30];

    int year = gps.date.year();

    if (gps.date.day() == 23 &&
      gps.date.month() == 3 &&
      (year == 2094 || year == 1994))
    {
      year = 1994;
    }

    snprintf(
      buffer,
      sizeof(buffer),
      "%04d-%02d-%02d %02d:%02d:%02d UTC",
      year,
      gps.date.month(),
      gps.date.day(),
      gps.time.hour(),
      gps.time.minute(),
      gps.time.second()
    );

    return String(buffer);
  }

  return "Unavailable";
}

void runGPSDemo()
{
  Serial.println("\n================================");
  Serial.println("TASK 1: GPS NMEA PARSING");
  Serial.println("RECORDED DATA - NOT LIVE GPS");
  Serial.println("================================");

  Serial.println("\nRecorded GGA sentence:");
  Serial.println(nmeaGGA);
  feedNMEA(nmeaGGA);

  Serial.println("Recorded RMC sentence:");
  Serial.println(nmeaRMC);
  feedNMEA(nmeaRMC);

  locationData.deviceId = deviceId;
  locationData.latitude = gps.location.lat();
  locationData.longitude = gps.location.lng();
  locationData.timestamp = getGPSTimestamp();
  locationData.gpsStatus = gps.location.isValid();
  locationData.fixQuality = -1;

  locationData.satellites = gps.satellites.isValid()
```

```
                        ? gps.satellites.value()
                        : -1;

    Serial.println("\n--- CLEAN LOCATION DATA ---");

    Serial.print("Device ID  : ");
    Serial.println(locationData.deviceId);

    if (locationData.gpsStatus)
    {
        Serial.print("Latitude   : ");
        Serial.println(locationData.latitude, 6);

        Serial.print("Longitude  : ");
        Serial.println(locationData.longitude, 6);
    }
    else
    {
        Serial.println("Latitude   : INVALID");
        Serial.println("Longitude  : INVALID");
    }

    Serial.print("Timestamp  : ");
    Serial.println(locationData.timestamp);

    Serial.print("GPS Status : ");
    Serial.println(locationData.gpsStatus
                   ? "VALID (RECORDED DATA)"
                   : "INVALID");

    Serial.print("Satellites : ");
    if (locationData.satellites >= 0)
        Serial.println(locationData.satellites);
    else
        Serial.println("Unavailable");

    Serial.print("HDOP       : ");
    if (gps.hdop.isValid())
        Serial.println(gps.hdop.hdop(), 2);
    else
        Serial.println("Unavailable");

    Serial.print("Checksum failures: ");
    Serial.println(gps.failedChecksum());

    Serial.println("GPS input is recorded test data.");
}

struct IMUData
{
    float ax;
    float ay;
    float az;
    float gx;
```

```
    float gy;
    float gz;
};

IMUData imu;

int testCase = 0;

void generateIMUData()
{
    switch (testCase)
    {
        case 0:
            imu = {0.02, 0.01, 0.98, 0.5, 0.2, 1.0};
            break;

        case 1:
            imu = {0.35, 0.15, 0.92, 12.0, 5.0, 3.0};
            break;

        case 2:
            imu = {1.20, 0.40, 0.80, 45.0, 20.0, 10.0};
            break;
    }
}

float calculateAccelerationMagnitude()
{
    return sqrtf(
        imu.ax * imu.ax +
        imu.ay * imu.ay +
        imu.az * imu.az
    );
}

void runIMUDemo()
{
    Serial.println("\n================================");
    Serial.println("TASK 2: IMU DATA PROCESSING");
    Serial.println("SIMULATED / TEST DATA");
    Serial.println("================================");

    generateIMUData();

    if (testCase == 0)
        Serial.println("Test case: STATIONARY");
    else if (testCase == 1)
        Serial.println("Test case: TILT / ROTATION");
    else
        Serial.println("Test case: MOVEMENT");

    Serial.printf("AX: %.2f g\n", imu.ax);
    Serial.printf("AY: %.2f g\n", imu.ay);
    Serial.printf("AZ: %.2f g\n", imu.az);
```

```
    Serial.printf("GX: %.2f deg/s\n", imu.gx);
    Serial.printf("GY: %.2f deg/s\n", imu.gy);
    Serial.printf("GZ: %.2f deg/s\n", imu.gz);

    Serial.printf(
        "Acceleration Magnitude: %.3f g\n",
        calculateAccelerationMagnitude()
    );

    Serial.println("IMU readings are simulated.");
}

void setup()
{
    Serial.begin(115200);
    delay(1000);

    Serial.println("\nESP32-S3 GPS + IMU MINI DEMO");
    Serial.println("GPS: RECORDED NMEA DATA");
    Serial.println("IMU: SIMULATED / TEST DATA");

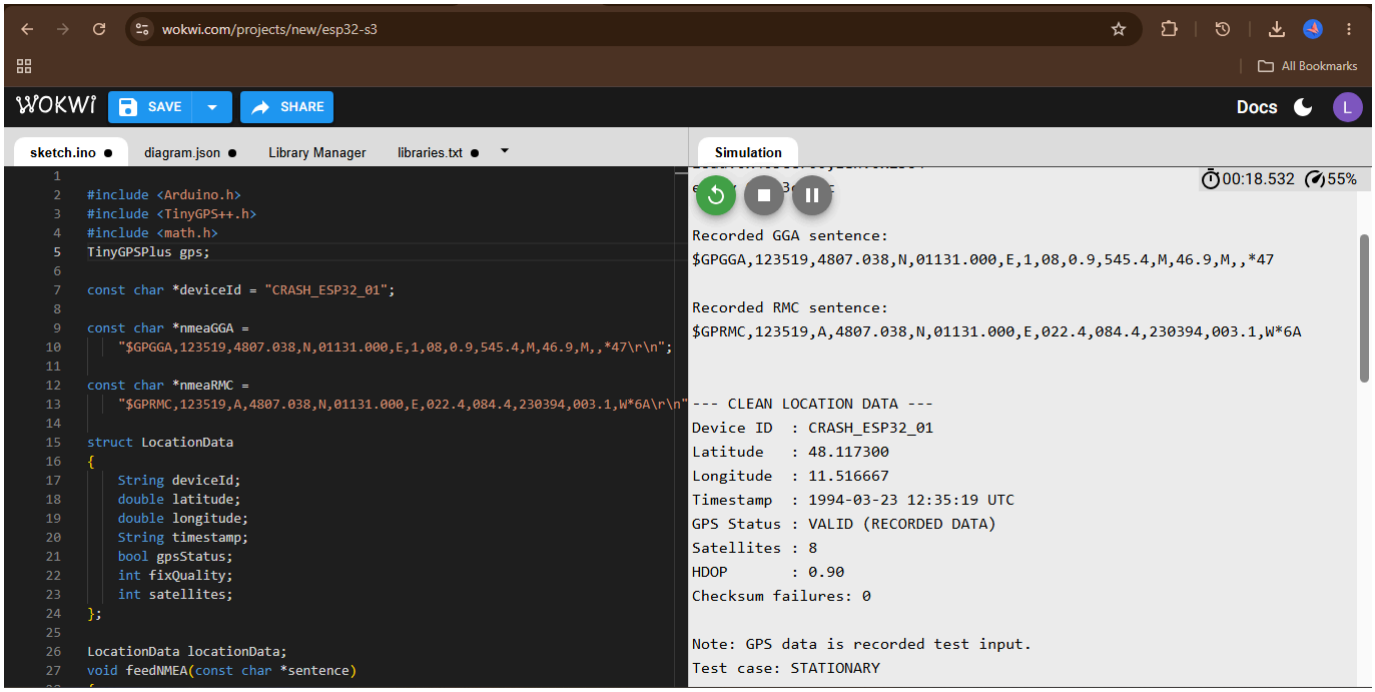
    runGPSDemo();
}

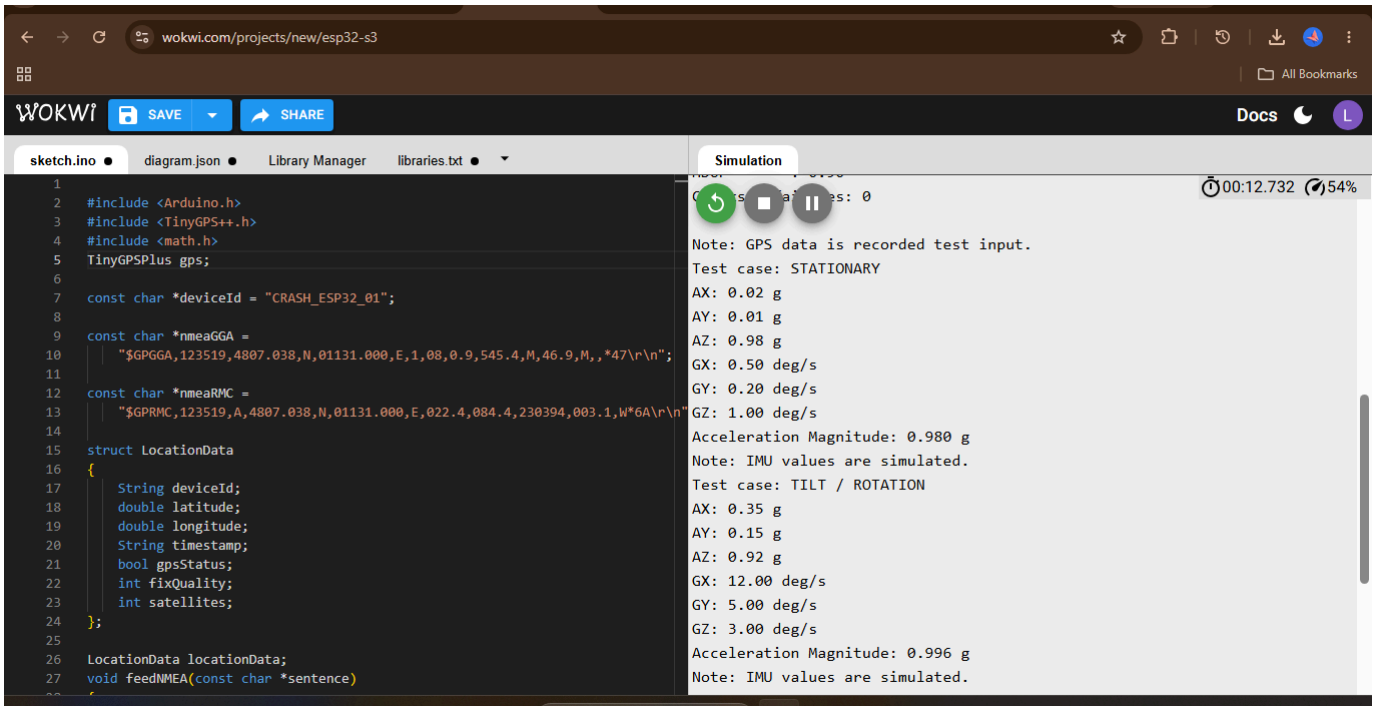
void loop()
{
    static unsigned long previousMillis = 0;
    const unsigned long interval = 3000;

    if (millis() - previousMillis >= interval)
    {
        previousMillis = millis();

        runIMUDemo();

        testCase = (testCase + 1) % 3;
    }
}
```





Wokwi Simulation Link : https://wokwi.com/projects/477477674875593729